Universidad Mariano Gálvez
Facultad de Ingeniería
Ingeniería en Sistemas de Información
Curso: Base de Datos
Catedrático: Ing. Edgar René Ornelis Hoíl

AutoAudit
Proyecto Final

MANUAL TÉCNICO

Fecha de Entrega: 07/11/2026

Grupo 4

Integrantes

Carnet	**Nombre**
1990-24-2538	María Luisa Sinaj Camey
1990-24-11866	Jordy Alejandro Najarro Cherec

Índice

1. Introducción 3
2. Descripción general del sistema 3
3. Arquitectura 5
4. Requisitos 7
5. Estructura del proyecto 9
6. Instalación y puesta en marcha 11
7. Configuración 16
8. Modelo de datos 17
9. Componentes de software 24
10. Función de auditoría 28
11. Seguridad 30
12. Pruebas y validación 30
13. Mantenimiento 36
14. Cronograma y calendario de control 37
Anexo C. Glosario 38
Anexo F. Manual de configuración y activación de auditoría por tabla 38

1. Introducción

1.1 Propósito del documento

Este manual describe el diseño interno, la instalación, la configuración y el mantenimiento de **AutoAudit**, un sistema de auditoría automatizada para bases de datos PostgreSQL. Está dirigido a administradores de bases de datos, desarrolladores y personal técnico que deban desplegar, modificar o dar soporte al sistema.

1.2 Alcance

El documento cubre la arquitectura de la solución, el modelo de datos con su diccionario, los componentes de software (aplicación Flask, capa de servicio y función de trigger), el procedimiento de instalación en Ubuntu, las medidas de seguridad implementadas, el plan de pruebas una sección de limitaciones conocidas detectadas durante la revisión del código, la verificación del cumplimiento de cada requisito del enunciado del proyecto y el calendario de control. El uso de la interfaz por parte de los usuarios finales se explica en el **Manual de usuario**.

1.3 Convenciones

- El texto con fuente monoespaciada indica comandos, nombres de archivos, tablas, columnas o código.
- Los recuadros azules son notas informativas; los amarillos señalan advertencias que deben atenderse; los verdes son consejos.
- Las rutas de archivos son relativas a la carpeta raíz del proyecto (AutoAudit-main/).

2. Descripción general del sistema

2.1 Objetivo

AutoAudit permite registrar de forma automática los cambios (inserciones, modificaciones y eliminaciones) que ocurren en las tablas de una base de datos PostgreSQL, sin modificar el código de las aplicaciones que usan esas tablas. El administrador elige desde una interfaz web qué tablas, columnas y operaciones auditar; el sistema genera dinámicamente los triggers necesarios y almacena en una bitácora quién hizo el cambio, cuándo, desde qué IP y cuáles fueron los valores anterior y nuevo.

2.2 Funcionalidades principales

- **Autenticación y roles:** inicio de sesión con usuario y contraseña; roles admin y auditor.

- **Descubrimiento de tablas y columnas:** lectura del catálogo de PostgreSQL (pg_class, information_schema.columns).
- **Configuración por tabla:** selección de columnas, operaciones (INSERT, UPDATE, DELETE) y estado habilitado/deshabilitado.
- **Generación dinámica de triggers:** creación o reemplazo de un trigger AFTER INSERT OR UPDATE OR DELETE por tabla que invoca una única función genérica.
- **Bitácora:** cabecera del evento con fila completa en JSONB y detalle por columna con valor anterior y nuevo.
- **Consulta y filtrado:** por usuario, tabla y columna (búsqueda parcial, sin distinguir mayúsculas).
- **Exportación a CSV:** descarga del historial (hasta 5000 filas).
- **Dashboard:** contadores de eventos, tablas auditadas y tablas disponibles, y actividad reciente.

2.3 Roles de usuario

Tabla 1. Roles definidos en la aplicación.

Rol	Módulos permitidos	Restricciones
admin	Dashboard, Configuración, Generador de triggers, Bitácora, Exportación CSV, Cerrar sesión.	Ninguna dentro de la aplicación.
auditor	Dashboard, Bitácora, Exportación CSV, Cerrar sesión.	Al intentar otra ruta se muestra el mensaje «No tienes permisos para acceder a este módulo.» y se redirige al Dashboard.

2.4 Tecnologías utilizadas

Tabla 2. Pila tecnológica.

Componente	Versión	Uso
Ubuntu	24.04 / 26.04 LTS	Sistema operativo del servidor (por ejemplo, dentro de una máquina virtual).
PostgreSQL	18	Motor de base de datos, triggers y función PL/pgSQL.
Python	3.12 o superior	Lenguaje de la aplicación.
Flask	>= 3.1, < 4	Framework web: rutas, sesiones, plantillas Jinja2.
psycopg (binary)	>= 3.2, < 4	Controlador PostgreSQL; consultas parametrizadas y sql.Identifier.
python-dotenv	>= 1.0, < 2	Carga de variables desde el archivo .env.

Werkzeug	Instalada con Flask	`check_password_hash` para validar contraseñas.
Bootstrap / Font Awesome	5.3.3 / 6.5.2 (CDN)	Estilos e iconos de la interfaz; se cargan desde internet en el navegador.

3. Arquitectura

3.1 Vista general

AutoAudit sigue una arquitectura de tres capas: el navegador del usuario, una aplicación Flask que contiene la lógica de configuración, y PostgreSQL, donde residen la configuración, la bitácora y los triggers. La captura de los cambios **no depende de la aplicación**: la realizan los triggers dentro de la propia base de datos, por lo que se registran incluso los cambios hechos desde `psql` o pgAdmin.

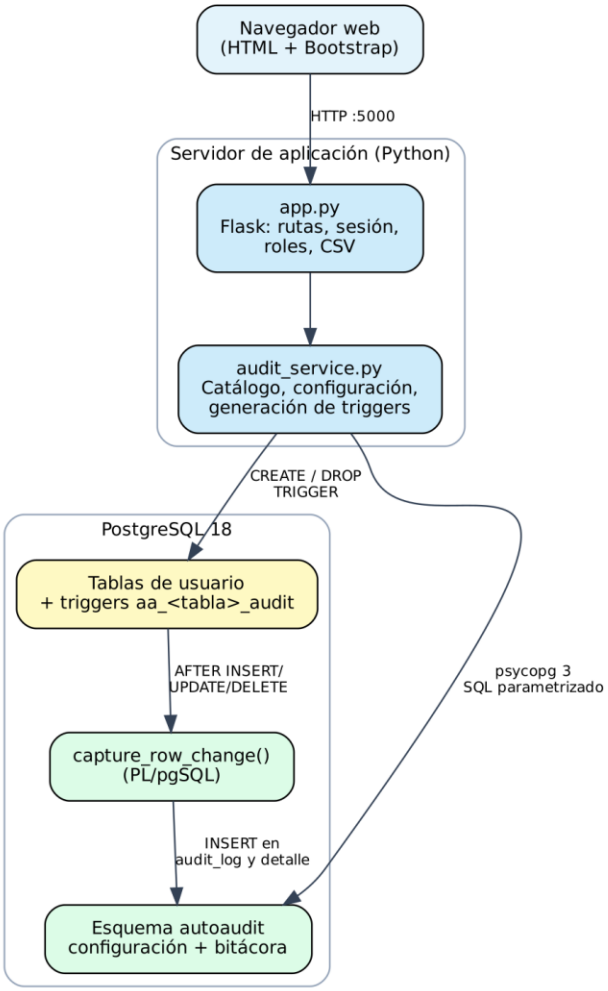


Figura 1. Arquitectura general de AutoAudit.

3.2 Responsabilidades por capa

Tabla 3. Capas del sistema.

Capa	Elementos	Responsabilidad
Presentación	Plantillas Jinja2 en `templates/`	Mostrar login, dashboard, configuración, generador de triggers y bitácora.
Aplicación	`app.py`	Rutas HTTP, sesión, control de acceso por rol, exportación CSV, manejo de errores.
Servicio	`audit_service.py`	Lectura del catálogo, persistencia de la configuración, creación/eliminación de triggers y consulta de la bitácora.
Datos	Esquema `autoaudit`, función `capture_row_change()`	Almacenar configuración y bitácora; capturar los cambios mediante triggers.

3.3 Flujo de captura de un cambio

Cuando se ejecuta una operación DML sobre una tabla auditada, el trigger `aa_<tabla>_audit` invoca la función `autoaudit.capture_row_change()`, que registra el evento siguiendo el flujo de la Figura 2.

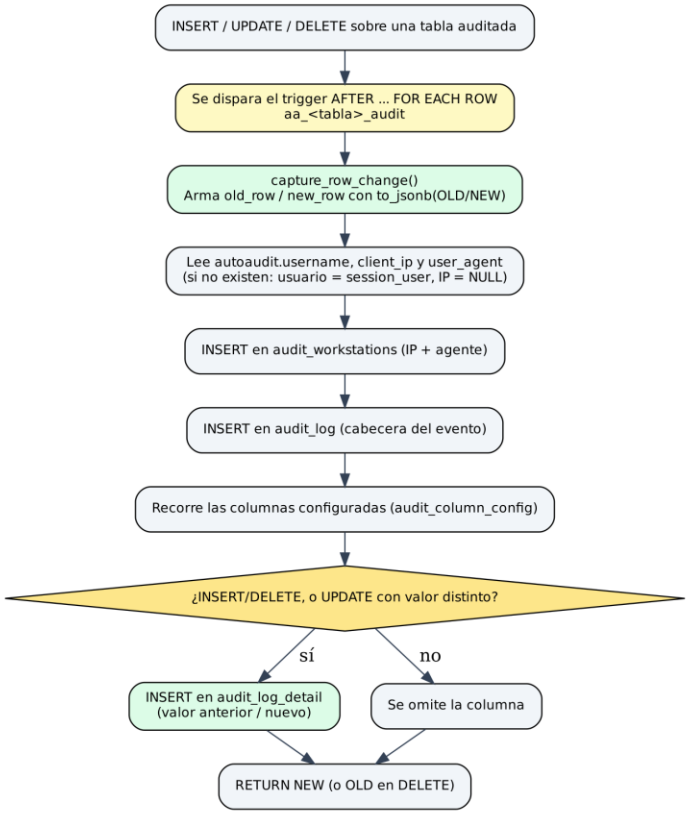


Figura 2. Flujo interno de la función de auditoría.

3.4 Decisiones de diseño

- **Una sola función genérica** (capture_row_change) atiende a todas las tablas; el trigger de cada tabla solo la invoca. Así no hay código duplicado por tabla.
- **Triggers AFTER ... FOR EACH ROW:** el registro se escribe una vez confirmada la validez de la operación sobre la fila.
- **Valores en JSONB:** permiten almacenar cualquier tipo de dato (texto, numérico, booleano, fecha) sin una columna por tipo.
- **Sin llave foránea hacia la tabla auditada:** la evidencia permanece aunque la fila original se elimine.
- **SQL seguro:** valores siempre con parámetros %s; nombres de esquema, tabla y trigger con psycopg.sql.Identifier.
- **SECURITY DEFINER con search_path fijo** (pg_catalog, autoaudit) en la función de auditoría.

4. Requisitos

4.1 Hardware recomendado

Tabla 4. Recursos sugeridos para una instalación de pruebas o académica.

Recurso	Mínimo recomendado
Procesador	2 vCPU
Memoria RAM	2 GB (4 GB si se usa pgAdmin dentro de la misma máquina virtual)
Almacenamiento	20 GB libres; la bitácora crece con cada cambio auditado

4.1.1 Equipo utilizado en el proyecto

El proyecto se desarrolló y se probó en una máquina virtual de Oracle VirtualBox ejecutada sobre el equipo físico siguiente:

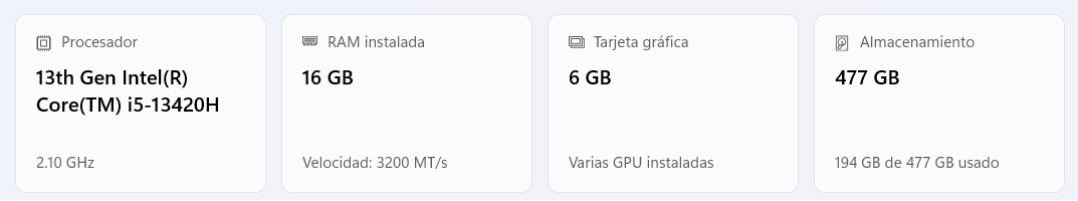


Figura 3. Especificaciones del equipo físico (anfitrión).

Tabla 5. Equipo físico (anfitrión).

Componente	Especificación
Equipo	HP Victus Gaming Laptop 15-fa2xxx

Procesador	13th Gen Intel Core i5-13420H, 2.10 GHz
Memoria RAM	16 GB instalados (15.6 GB utilizables), 3200 MT/s
Tarjeta gráfica	NVIDIA GeForce RTX 4050 Laptop GPU (6 GB) e Intel UHD Graphics (128 MB)
Almacenamiento	477 GB (194 GB usados al momento de la captura)
Tipo de sistema	Sistema operativo de 64 bits, procesador x64

Tabla 6. Entorno virtual de desarrollo y pruebas.

Elemento del entorno virtual	Valor
Software de virtualización	Oracle VirtualBox
Sistema operativo invitado	Ubuntu 24.04 LTS
Base de datos	PostgreSQL 18

4.2 Software

- Ubuntu 24.04 LTS o 26.04 LTS (u otra versión indicada por el catedrático).
- PostgreSQL 18 con acceso al cliente `psql`.
- Python 3.12 o superior con `venv` y `pip`.
- Git (opcional) para clonar el repositorio.
- Navegador web actualizado, con acceso a `cdn.jsdelivr.net` y `cdnjs.cloudflare.com` para cargar estilos e iconos.
- pgAdmin (opcional) para inspeccionar la base de datos de forma gráfica.

4.3 Puertos

Tabla 7. Puertos utilizados.

Puerto	Servicio	Observación
5000/TCP	Aplicación Flask	Por defecto escucha solo en `127.0.0.1` (definido en `app.py`).
5432/TCP	PostgreSQL	La aplicación se conecta mediante `DATABASE_URL`.

5. Estructura del proyecto

```
AutoAudit-main/
├── app.py                Aplicación Flask: rutas, sesión y roles
├── audit_service.py      Catálogo, configuración y generación de triggers
├── requirements.txt      Dependencias de Python
├── .env.example          Plantilla de variables de entorno
├── .gitignore            Exclusiones de Git (.env, .venv, respaldos)
├── README.md             Guía rápida
├── docs/
│   └── modelo_datos.md   Descripción del modelo de datos
├── sql/
│   ├── 01_schema.sql     Esquema autoaudit, tablas y función de trigger
│   └── 02_demo.sql       Tabla de demostración empleados_demo
├── templates/
│   ├── base.html         Plantilla base (barra lateral y superior)
│   ├── login.html        Inicio de sesión
│   ├── index.html        Dashboard
│   ├── configure.html    Configuración de auditoría
│   ├── triggers.html     Generador de triggers
│   └── logs.html         Bitácora
└── tests/
    └── test_plan.md      Plan de pruebas de validación
```

Las capturas siguientes muestran el proyecto desplegado en la máquina virtual (carpeta `~/Descargas/AutoAudit/AutoAudit`): el listado de archivos y el de directorios.

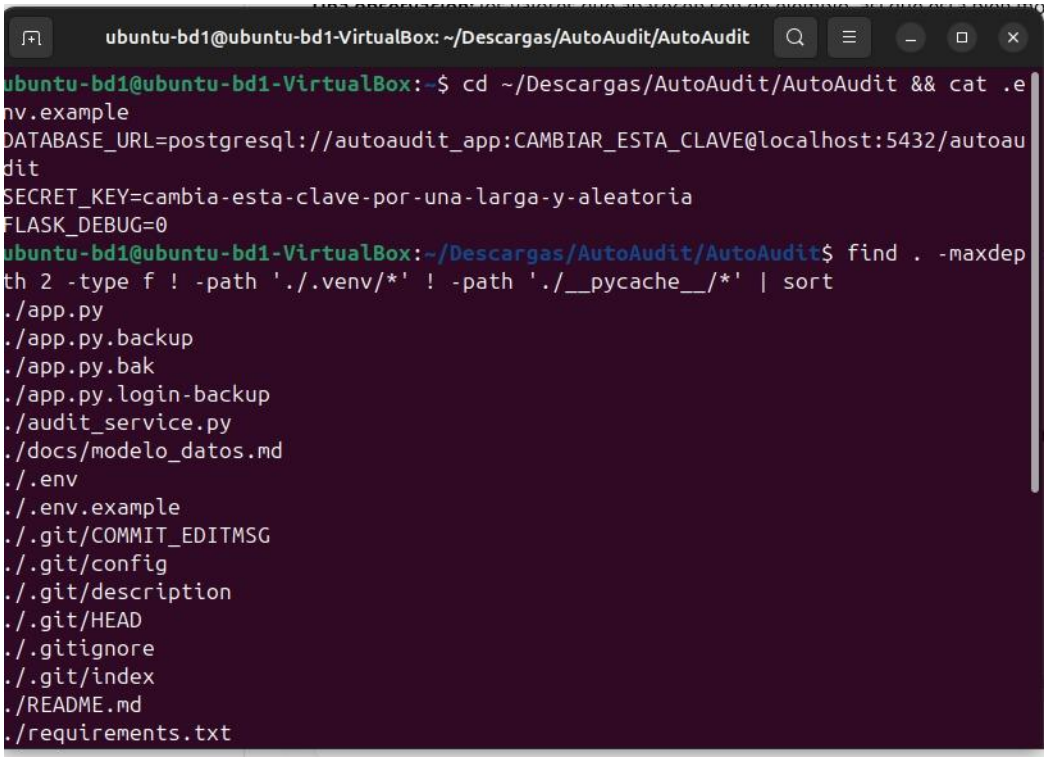


Figura 4. Listado de archivos del proyecto en la máquina virtual (comando find).

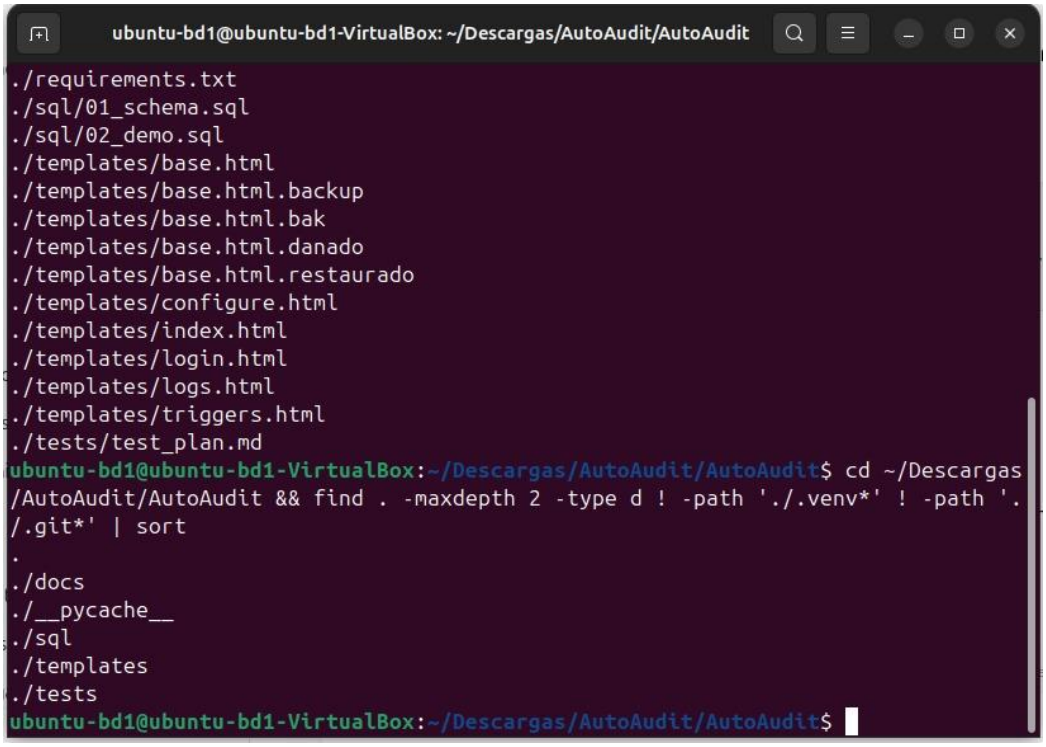


Figura 5. Archivos de plantillas y directorios del proyecto (docs, sql, templates, tests).

6. Instalación y puesta en marcha

Los pasos siguientes se probaron conceptualmente contra el código fuente. Se ejecutan con un usuario con privilegios `sudo` en Ubuntu.

6.1 Preparar el sistema

```
sudo apt update
sudo apt install -y python3 python3-venv python3-pip git
```

6.2 Instalar PostgreSQL 18

Ubuntu 26.04 incluye PostgreSQL 18 en su repositorio oficial. En Ubuntu 24.04 el paquete `postgresql` corresponde a una versión anterior, por lo que se debe agregar el repositorio oficial de PostgreSQL (PGDG):

```
sudo apt install -y postgresql-common
sudo /usr/share/postgresql-common/pgdg/apt.postgresql.org.sh -y
sudo apt install -y postgresql-18
sudo systemctl enable --now postgresql
psql --version
```

6.3 Crear el usuario y la base de datos

```
sudo -u postgres psql
```

Dentro de `psql` ejecute (reemplace la contraseña por una clave segura):

```
CREATE USER autoaudit_app WITH PASSWORD 'CAMBIAR_ESTA_CLAVE';
CREATE DATABASE autoaudit OWNER autoaudit_app;
\q
```

El usuario `autoaudit_app` queda como propietario de la base de datos; esto le permite crear el esquema `autoaudit`, la tabla de demostración y los triggers sobre tablas que él mismo cree.

6.4 Obtener el proyecto

```
# Opción A: desde el archivo comprimido
unzip AutoAudit-main.zip && cd AutoAudit-main

# Opción B: desde el repositorio remoto
git clone <URL_DEL_REPOSITORIO> && cd AutoAudit-main
```

6.5 Entorno virtual y dependencias

```
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

6.6 Variables de entorno

```
cp .env.example .env
nano .env
```

Edite DATABASE_URL con las credenciales reales y cambie SECRET_KEY (véase la sección 7). La plantilla .env.example contiene las tres variables que el sistema utiliza:

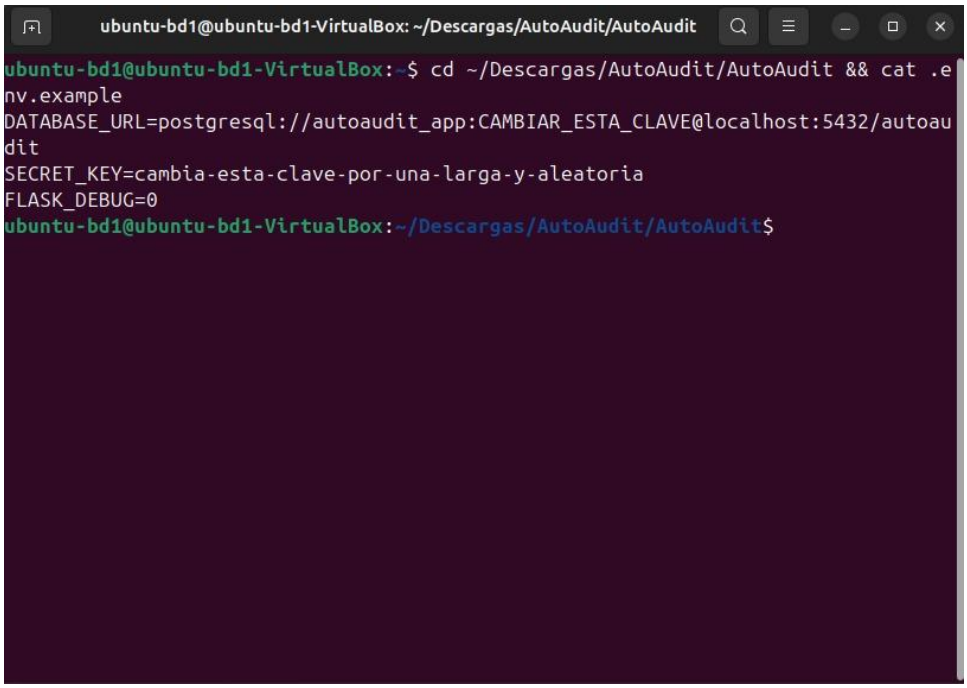


Figura 6. Contenido de la plantilla .env.example.

6.7 Crear el esquema y los datos de demostración

```
psql "postgresql://autoaudit_app:CAMBIAR_ESTA_CLAVE@localhost:5432/autoaudit" -
f sql/01_schema.sql
psql "postgresql://autoaudit_app:CAMBIAR_ESTA_CLAVE@localhost:5432/autoaudit" -
f sql/02_demo.sql
```

6.8 Crear los usuarios de la aplicación (paso obligatorio)

Primero genere el hash de cada contraseña con Werkzeug (con el entorno virtual activo):

```
python3 -c "from werkzeug.security import generate_password_hash;
print(generate_password_hash('MiClaveSegura123'))"
```

Copie el resultado en la sentencia `INSERT` del Anexo A y ejecútela con `psql` o desde pgAdmin.

6.9 Iniciar la aplicación

```
source .venv/bin/activate
flask --app app run
```

El equipo inició la aplicación con `flask --app app run`, equivalente a ejecutar `python app.py`. La terminal informa que Flask sirve la aplicación en `http://127.0.0.1:5000` con el modo de depuración desactivado y muestra el registro de peticiones (`GET /triggers`, `GET /exportar`, `POST /login`, etc.). El aviso en rojo indica que se trata del servidor de desarrollo de Flask.

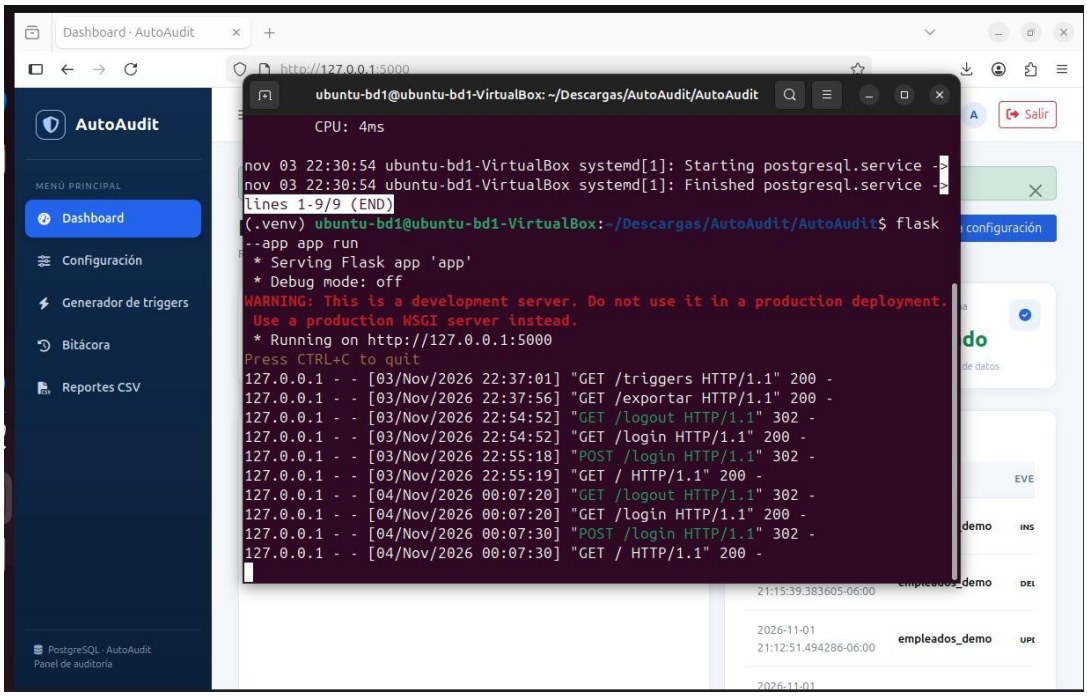


Figura 7. Aplicación en ejecución: terminal con el servidor Flask y el Dashboard en el navegador.

Al abrir `http://127.0.0.1:5000` se muestra la pantalla de inicio de sesión.

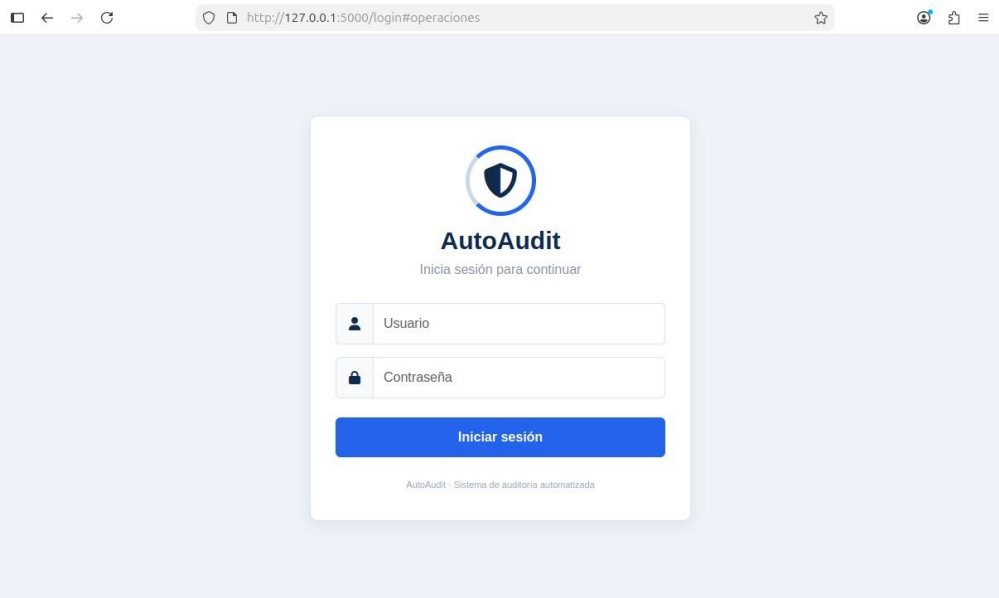


Figura 8. Pantalla de inicio de sesión de AutoAudit.

6.10 Verificación de la instalación

Tabla 8. Lista de comprobación posterior a la instalación.

Verificación	Resultado esperado
psql ... -c "\dt autoaudit.*"	Se listan las tablas audit_table_config, audit_column_config, audit_event_type, audit_users, audit_workstations, audit_log, audit_log_detail y app_users.
psql ... -c "\df autoaudit.*"	Aparece la función capture_row_change.
Abrir http://127.0.0.1:5000	Se muestra el formulario de inicio de sesión.
Ingresar con un usuario admin	Se muestra el Dashboard y la tabla public.empleados_demo aparece en «Tablas detectadas».

En pgAdmin se registró el servidor (base autoaudit, usuario autoaudit_app) para revisar visualmente el esquema autoaudit. La captura siguiente muestra sus 8 tablas (incluida app_users) y la función de trigger capture_row_change() en «Trigger Functions».

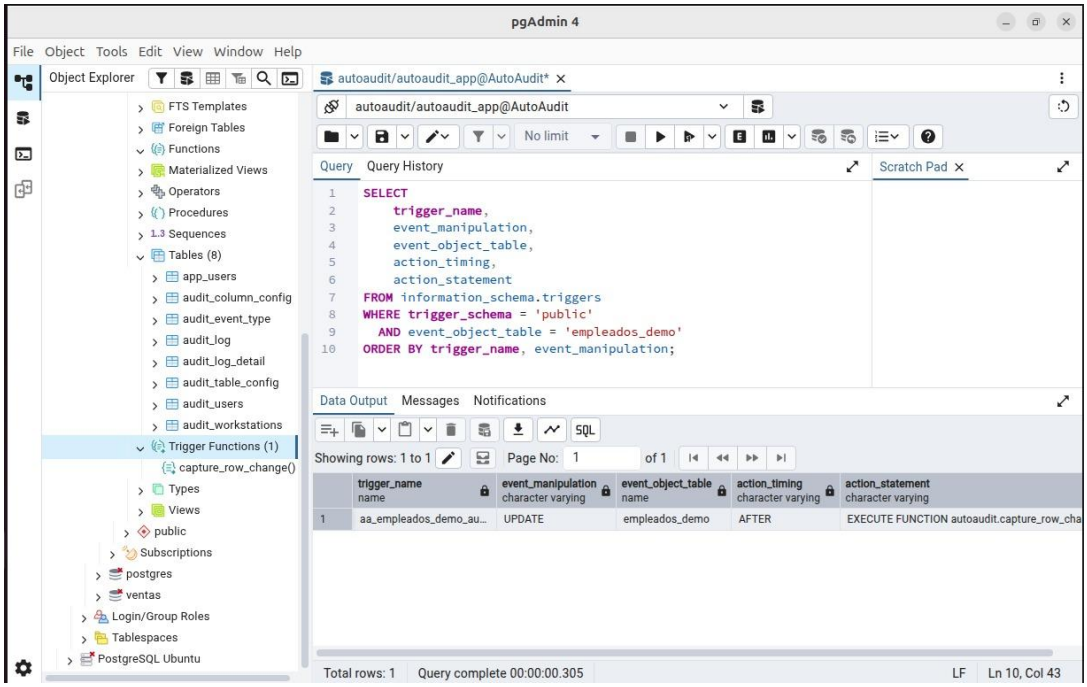


Figura 9. pgAdmin: tablas del esquema autoaudit y función capture_row_change().

6.11 Despliegue de los triggers de auditoría

Los triggers no se instalan con los scripts SQL: se **generan dinámicamente** por tabla a partir de la configuración. El procedimiento de despliegue es el siguiente:

1. Confirmar que la función genérica existe: `\df autoaudit.capture_row_change`.
2. Iniciar sesión como `admin`, abrir **Configuración**, elegir la tabla a auditar, marcar columnas y operaciones, dejar «Auditoría habilitada» y pulsar **Guardar y generar trigger**.
3. Comprobar que el trigger fue creado con la consulta siguiente.
4. Si un trigger falta o fue eliminado manualmente, abrir **Generador de triggers** y pulsar **Generar / actualizar trigger** en la tabla correspondiente.
5. Para retirar la auditoría de una tabla, desmarcar «Auditoría habilitada» y guardar; el trigger se elimina y la configuración y el historial se conservan.

```
SELECT tgrelid::regclass AS tabla, tgname, tgenabled
FROM pg_trigger
WHERE tgname LIKE 'aa\_%' AND NOT tgisinternal;
```

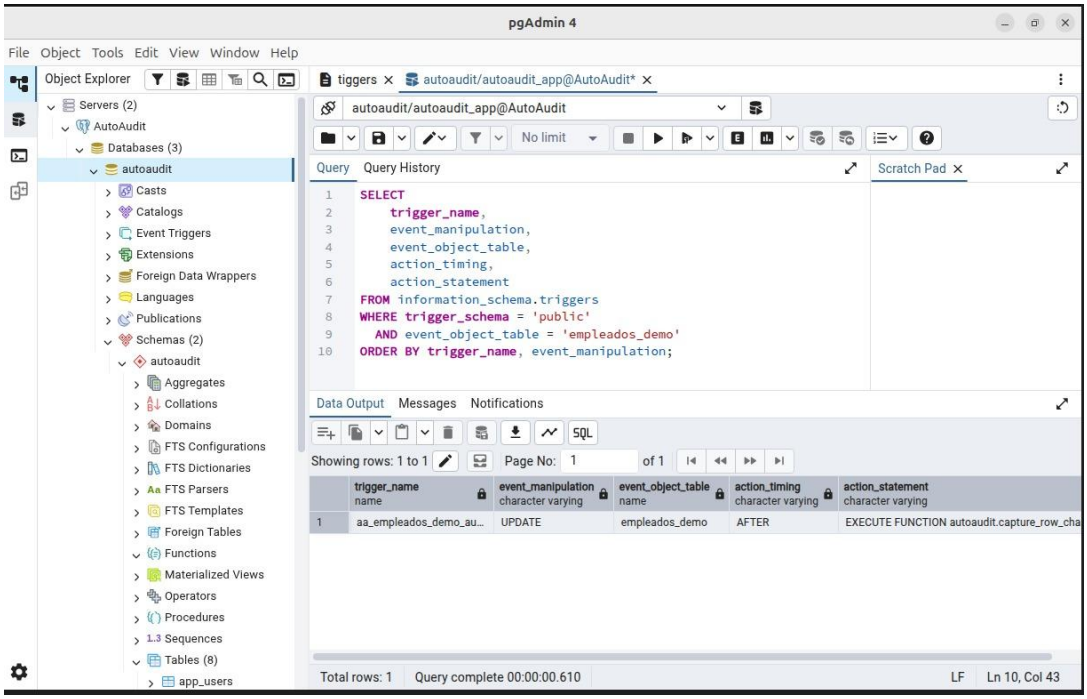


Figura 10. pgAdmin: consulta a information_schema.triggers para la tabla empleados_demo.

Cada trigger atiende **exclusivamente una tabla origen**; todos invocan la misma función genérica, que identifica la tabla mediante `TG_TABLE_SCHEMA` y `TG_TABLE_NAME`.

7. Configuración

7.1 Archivo. env

Tabla 9. Variables de entorno.

Variable	Ejemplo	Descripción
`DATABASE_URL`	`postgresql://autoaudit_app:clave@localhost:5432/autoaudit`	Cadena de conexión a PostgreSQL. Si no está definida, la aplicación lanza «Configura DATABASE_URL en el archivo .env».
`SECRET_KEY`	Cadena larga y aleatoria	Firma las cookies de

		sesión. Si falta, se usa un valor de desarrollo inseguro (development-only-change-me).
FLASK_DEBUG	0	Con valor 1 activa el modo depuración. Debe ser 0 fuera del desarrollo.

7.2 Dirección y puerto de la aplicación

El servidor se inicia al final de app.py con app.run(host="127.0.0.1", port=5000, ...). Para cambiar el puerto o permitir acceso desde otras máquinas debe modificarse esa línea; en ese caso conviene colocar la aplicación detrás de un servidor WSGI y un proxy con HTTPS (véase la sección 11.3).

8. Modelo de datos

8.1 Diagrama entidad-relación

Todas las tablas del sistema residen en el esquema autoaudit. Las tablas en celeste intenso almacenan la configuración, las celeste claro la bitácora, las grises son catálogos sin relación de llave foránea y la amarilla corresponde a los usuarios de la aplicación (app_users), que debe crearse manualmente.

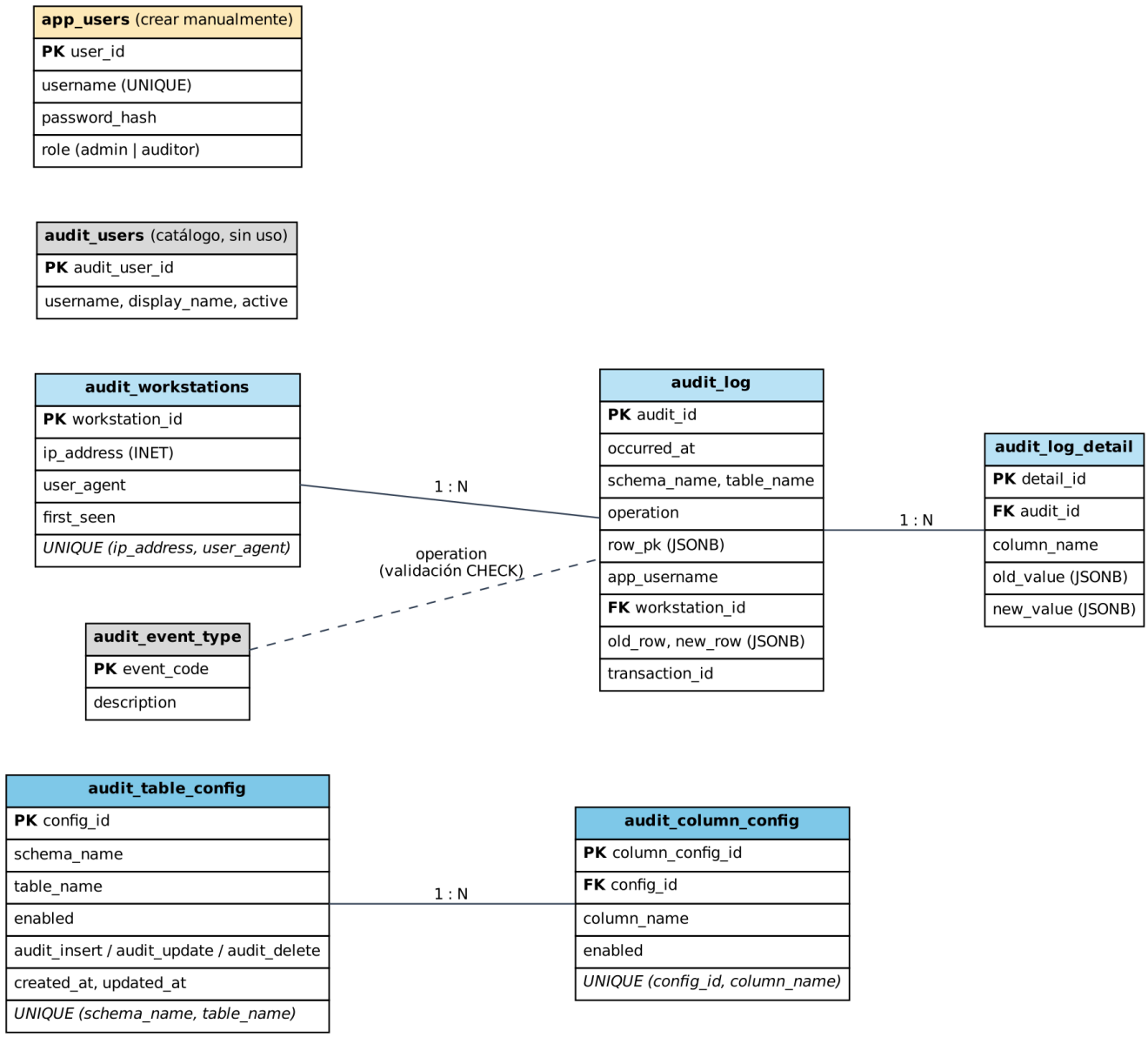


Figura 11. Diagrama entidad-relación del esquema autoaudit.

8.2 Relaciones

- audit_table_config 1 : N audit_column_config (eliminación en cascada).
- audit_workstations 1 : N audit_log (si se elimina la estación, workstation_id queda en NULL).
- audit_log 1 : N audit_log_detail (eliminación en cascada).
- audit_log no tiene llave foránea hacia las tablas auditadas, para que la evidencia sobreviva a un DELETE.
- audit_event_type documenta las operaciones válidas; la validación efectiva la hace la restricción CHECK de audit_log.operation.

8.3 Diccionario de datos

autoaudit.audit_table_config

Configuración de auditoría por tabla.

Tabla 10. Diccionario de audit_table_config.

Columna	Tipo	Nulo	Clave / restricción	Descripción
config_id	BIGSERIAL	No	PK	Identificador de la configuración.
schema_name	TEXT	No	UNIQUE (con table_name)	Esquema de la tabla auditada.
table_name	TEXT	No	UNIQUE (con schema_name)	Nombre de la tabla auditada.
enabled	BOOLEAN	No	DEFAULT TRUE	Indica si la auditoría está activa.
audit_insert	BOOLEAN	No	DEFAULT TRUE	Auditar inserciones.
audit_update	BOOLEAN	No	DEFAULT TRUE	Auditar actualizaciones.
audit_delete	BOOLEAN	No	DEFAULT TRUE	Auditar eliminaciones.
created_at	TIMESTAMPTZ	No	DEFAULT CURRENT_TIMESTAMP	Fecha de creación.
updated_at	TIMESTAMPTZ	No	DEFAULT CURRENT_TIMESTAMP	Fecha de la última modificación.

autoaudit.audit_column_config

Columnas que se auditan en cada tabla configurada.

Tabla 11. Diccionario de audit_column_config.

Columna	Tipo	Nulo	Clave / restricción	Descripción
column_config_id	BIGSERIAL	No	PK	Identificador.
config_id	BIGINT	No	FK → audit_table_config, ON DELETE CASCADE	Configuración de tabla a la que pertenece.

column_name	TEXT	No	UNIQUE (con config_id)	Nombre de la columna auditada.
enabled	BOOLEAN	No	DEFAULT TRUE	Columna activa.

autoaudit.audit_event_type

Catálogo de operaciones soportadas; contiene tres filas: INSERT, UPDATE y DELETE.

Tabla 12. Diccionario de audit_event_type.

Columna	Tipo	Nulo	Clave / restricción	Descripción
event_code	TEXT	No	PK, CHECK (INSERT, UPDATE, DELETE)	Código de la operación.
description	TEXT	No	—	Descripción en español.

autoaudit.audit_users

Catálogo previsto para usuarios de aplicación. La versión actual no lo utiliza.

Tabla 13. Diccionario de audit_users.

Columna	Tipo	Nulo	Clave / restricción	Descripción
audit_user_id	BIGSERIAL	No	PK	Identificador.
username	TEXT	No	UNIQUE	Nombre de usuario.
display_name	TEXT	Sí	—	Nombre para mostrar.
active	BOOLEAN	No	DEFAULT TRUE	Usuario activo.

autoaudit.audit_workstations

Estaciones de trabajo desde las que se originan los cambios.

Tabla 14. Diccionario de audit_workstations.

Columna	Tipo	Nulo	Clave / restricción	Descripción
workstation_id	BIGSERIAL	No	PK	Identificador.
ip_address	INET	Sí	UNIQUE (con user_agent)	Dirección IP del cliente.

user_agent	TEXT	Sí	UNIQUE (con ip_address)	Agente de usuario del cliente.
first_seen	TIMESTAMPTZ	No	DEFAULT CURRENT_TIMESTAMP	Primera vez que se registró.

autoaudit.audit_log

Cabecera de cada evento auditado.

Tabla 15. Diccionario de audit_log.

Columna	Tipo	Nulo	Clave / restricción	Descripción
audit_id	BIGSERIAL	No	PK	Identificador del evento.
occurred_at	TIMESTAMPTZ	No	DEFAULT clock_timestamp()	Fecha y hora exactas del evento.
schema_name	TEXT	No	—	Esquema de la tabla modificada.
table_name	TEXT	No	—	Tabla modificada.
operation	TEXT	No	CHECK (INSERT, UPDATE, DELETE)	Operación realizada.
row_pk	JSONB	Sí	—	Identificador de la fila: ctid; en UPDATE, ctid_old y ctid_new.
app_username	TEXT	No	DEFAULT 'desconocido'	Usuario que hizo el cambio.
workstation_id	BIGINT	Sí	FK → audit_workstations, ON DELETE SET NULL	Estación de origen.
old_row	JSONB	Sí	—	Fila completa antes del cambio (NULL en INSERT).

new_row	JSONB	Sí	—	Fila completa después del cambio (NULL en DELETE).
transaction_id	BIGINT	No	DEFAULT txid_current()	Transacción que originó el evento.

autoaudit.audit_log_detail

Valores anterior y nuevo por cada columna auditada.

Tabla 16. Diccionario de audit_log_detail.

Columna	Tipo	Nulo	Clave / restricción	Descripción
detail_id	BIGSERIAL	No	PK	Identificador.
audit_id	BIGINT	No	FK → audit_log, ON DELETE CASCADE	Evento al que pertenece.
column_name	TEXT	No	—	Columna auditada.
old_value	JSONB	Sí	—	Valor anterior.
new_value	JSONB	Sí	—	Valor nuevo.

autoaudit.app_users (a crear con el Anexo A)

Usuarios que pueden iniciar sesión en la interfaz web. Su estructura se deduce de la consulta de app.py; los tipos y restricciones son los propuestos en el Anexo A.

Tabla 17. Diccionario de app_users.

Columna	Tipo	Nulo	Clave / restricción	Descripción
user_id	BIGSERIAL	No	PK	Identificador (se guarda en sesión).
username	TEXT	No	UNIQUE	Nombre de usuario para el login.
password_hash	TEXT	No	—	Hash de Werkzeug de la contraseña.
role	TEXT	No	CHECK ('admin', 'auditor')	Rol que determina los módulos permitidos.

8.4 Índices

Tabla 18. Índices definidos.

Índice	Tabla y columnas	Propósito
idx_audit_log_time	audit_log (occurred_at DESC)	Ordenar la bitácora por fecha descendente.
idx_audit_log_user	audit_log (app_username)	Filtrar por usuario.
idx_audit_log_table	audit_log (schema_name, table_name)	Filtrar por tabla.
idx_audit_detail_column	audit_log_detail (column_name)	Filtrar por columna.

8.5 Ejemplo de datos almacenados

Las capturas siguientes muestran el contenido real de la bitácora tras las pruebas del equipo, consultado desde pgAdmin. La primera corresponde a audit_log (una fila por evento): cuatro eventos sobre public.empleados_demo (INSERT, UPDATE, DELETE e INSERT). En row_pk se observa el ctid de la fila y, en el UPDATE, ctid_old y ctid_new.

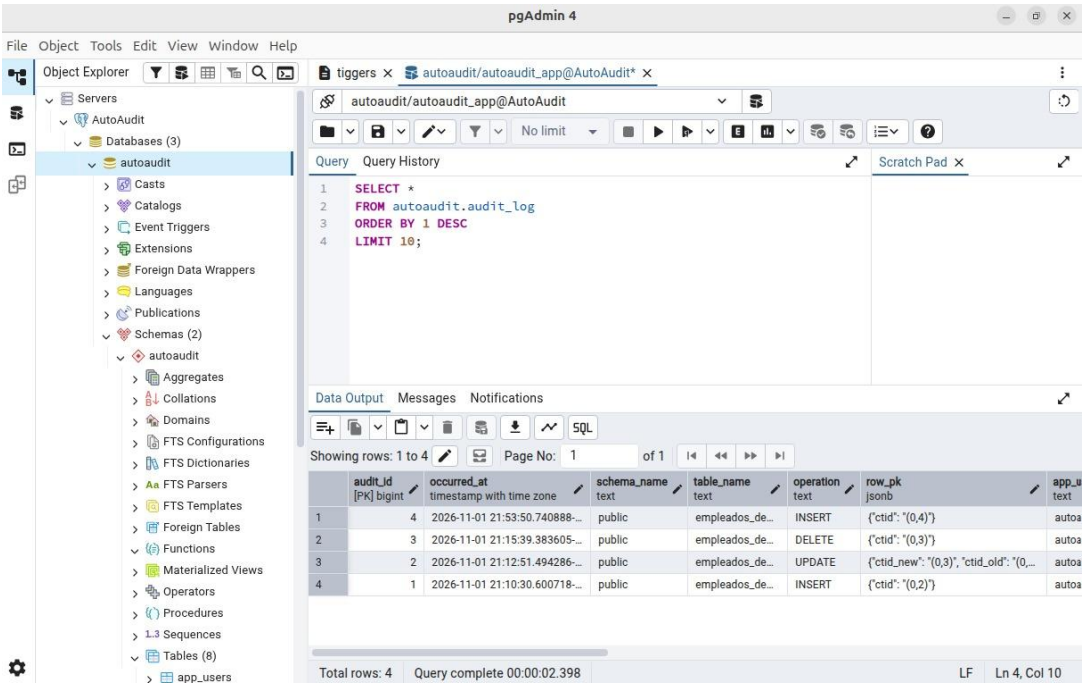


Figura 12. pgAdmin: consulta a autoaudit.audit_log.

La segunda corresponde a audit_log_detail (una fila por columna auditada). Los valores se guardan en JSONB: los textos aparecen entre comillas, los números sin ellas y [null] indica que no había valor (por ejemplo, el valor anterior en un INSERT).

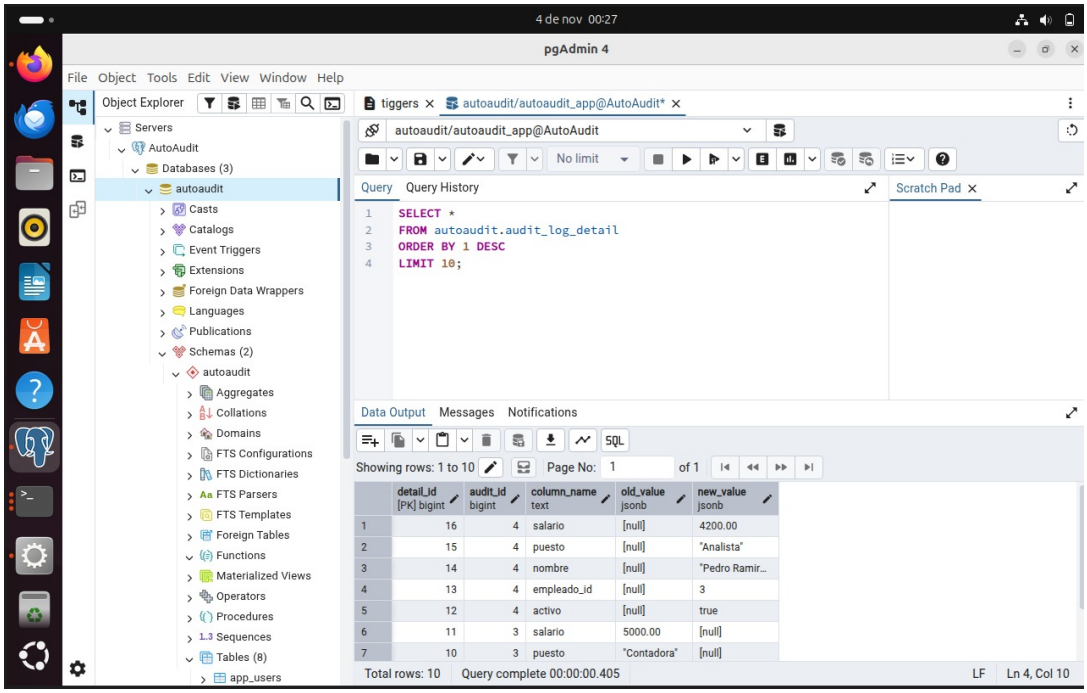


Figura 13. pgAdmin: consulta a autoaudit.audit_log_detail.

9. Componentes de software

9.1 app.py — aplicación Flask

Tabla 19. Rutas de la aplicación.

Ruta	Método	Rol	Descripción
/login	GET, POST	Público	Valida usuario y contraseña contra autoaudit.app_users; guarda user_id, username y role en la sesión.
/logout	GET	Todos	Limpia la sesión y redirige al login.
/ (index)	GET	admin, auditor	Dashboard: totales de eventos y de tablas auditadas, tablas detectadas y 8 eventos recientes.
/configurar	GET, POST	admin	GET: carga columnas y configuración de una tabla. POST: guarda la configuración y despliega el trigger en una sola transacción.
/triggers	GET	admin	Lista las configuraciones con operaciones, columnas y una vista previa del SQL del trigger.
/triggers/generar	POST	admin	Crea o reemplaza el trigger de una tabla ya configurada.

/bitacora	GET	admin, auditor	Consulta de la bitácora con filtros usuario, tabla y columna (hasta 500 filas).
/exportar	GET	admin, auditor	Descarga autoaudit_bitacora.csv (hasta 5000 filas) respetando los filtros.

Mecanismos transversales

- verificar_sesion (@app.before_request): permite sin sesión únicamente login y static; si no hay sesión redirige al login. Para el rol auditor solo permite los endpoints index, logs, export_csv y logout.
- db_page: decorador que captura cualquier excepción, la registra en el log de Flask, muestra «Error: …» y redirige al Dashboard. No se aplica a las rutas login ni logout.
- get_conn(): abre una conexión psycopg con row_factory=dict_row usando DATABASE_URL.
- inject_now: expone la variable year a las plantillas.
- attach_audit_context (@app.after_request): actualmente no realiza ninguna acción; el comentario indica que el contexto de auditoría debería establecerse en cada operación de escritura (véase la sección 13).

9.2 audit_service.py — capa de servicio

Tabla 20. Funciones de la capa de servicio.

Función	Parámetros	Comportamiento
list_user_tables	conn	Devuelve esquema y nombre de las tablas ordinarias y particionadas (relkind r, p), excluyendo pg_catalog, information_schema, autoaudit y pg_toast*.
list_columns	conn, schema_name, table_name	Devuelve nombre, tipo y nulabilidad de las columnas desde information_schema.columns, en orden de posición.
get_config	conn, schema_name, table_name	Devuelve la configuración de la tabla y el estado de sus columnas; (None, []) si no existe.
save_config	conn, schema_name, table_name,	Valida las columnas contra el catálogo, exige al menos una columna válida, hace UPSERT de la

	columns, operations, enabled	configuración y reemplaza las columnas configuradas.
deploy_trigger	conn, schema_name, table_name	Si la auditoría está deshabilitada o sin operaciones, elimina el trigger. En caso contrario, lo elimina y lo recrea con los eventos configurados.
query_logs	conn, username, table_name, column_name, limit=500	Une audit_log, audit_log_detail y audit_workstations; filtra con ILIKE parcial y ordena por fecha descendente.

9.3 Plantillas

Tabla 21. Plantillas Jinja2.

Archivo	Contenido
base.html	Estructura común: barra lateral con menú (las opciones Configuración y Generador de triggers solo se muestran al admin), barra superior con usuario, rol y botón «Salir», y área de mensajes.
login.html	Formulario de inicio de sesión independiente de base.html.
index.html	Tarjetas de métricas, tabla de tablas detectadas y actividad reciente.
configure.html	Selector de tabla, casillas de columnas y operaciones, interruptor «Auditoría habilitada».
triggers.html	Tarjetas por configuración con operaciones, columnas, SQL del trigger y botón para regenerarlo.
logs.html	Filtros y tabla de resultados con botón «Exportar CSV».

9.4 Flujo de configuración de una tabla

1. El administrador abre /configurar, elige esquema y tabla y pulsa «Cargar tabla» (GET con schema_name y table_name).
2. La aplicación consulta list_columns y get_config y muestra las columnas con las casillas marcadas según la configuración guardada; si la tabla nunca se configuró, todo aparece marcado.
3. Al pulsar «Guardar y generar trigger» (POST) se ejecuta save_config, que valida columnas y persiste la configuración.
4. En la misma conexión se ejecuta deploy_trigger, que crea o elimina el trigger.

5. Se confirma la transacción (`conn.commit()`) y se muestra el mensaje de resultado. Si ocurre un error, no se confirma nada.

9.5 SQL generado dinámicamente

Para una tabla con las tres operaciones habilitadas, `deploy_trigger` ejecuta el equivalente a:

```
DROP TRIGGER IF EXISTS "aa_empleados_demo_audit" ON
"public"."empleados_demo";

CREATE TRIGGER "aa_empleados_demo_audit"
AFTER INSERT OR UPDATE OR DELETE ON "public"."empleados_demo"
FOR EACH ROW
EXECUTE FUNCTION autoaudit.capture_row_change();
```

El nombre del trigger sigue el patrón `aa_<tabla>_audit`. Los identificadores se construyen con `sql.Identifier`, por lo que nunca se concatenan cadenas recibidas del usuario. PostgreSQL limita los identificadores a 63 bytes; con nombres de tabla muy largos el nombre del trigger se truncaría.

La Figura siguiente muestra la pantalla **Generador de triggers** de la instalación del equipo, con la tabla `public.empleados_demo` configurada y el SQL generado.

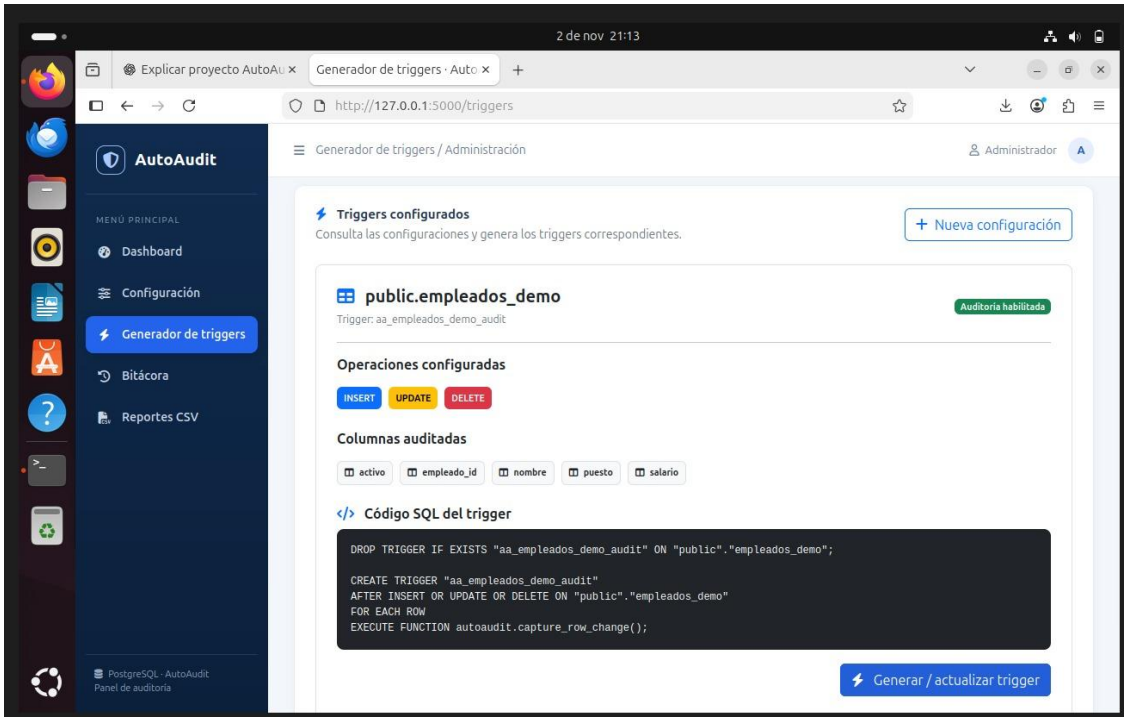


Figura 14. Pantalla «Generador de triggers» con el trigger aa_empleados_demo_audit.

10. Función de auditoría capture_row_change()

10.1 Características

- Lenguaje PL/pgSQL; retorna trigger.
- SECURITY DEFINER: se ejecuta con los privilegios del rol que creó la función (quien corrió 01_schema.sql).
- SET search_path = pg_catalog, autoaudit: evita ataques por manipulación de search_path.
- Devuelve NEW (INSERT, UPDATE) u OLD (DELETE).

10.2 Pasos que ejecuta

1. Según TG_OP, construye old_row y new_row con to_jsonb(OLD) y to_jsonb(NEW), y row_pk con el ctid de la fila.
2. Obtiene el usuario desde current_setting('autoaudit.username', true); si está vacío usa session_user.
3. Obtiene la IP desde autoaudit.client_ip (convertida a INET, NULL si es inválida) y el agente desde autoaudit.user_agent.
4. Inserta (o actualiza) la estación en audit_workstations y recupera workstation_id.
5. Inserta la cabecera en audit_log y recupera audit_id.
6. Recorre las columnas habilitadas de audit_column_config para la tabla; por cada una extrae el valor anterior y el nuevo del JSONB.
7. Inserta en audit_log_detail si la operación es INSERT o DELETE, o si en un UPDATE el valor cambió (IS DISTINCT FROM).

10.3 Variables de contexto

Tabla 22. Variables que la función lee con current_setting(..., true).

Variable de sesión	Contenido	Valor si no está definida
autoaudit.username	Usuario de aplicación que realiza el cambio.	session_user de PostgreSQL
autoaudit.client_ip	IP del cliente.	NULL
autoaudit.user_agent	Agente de usuario del navegador o cliente.	NULL

10.4 Consulta del código de la función en la base de datos

El código instalado de la función puede obtenerse con pg_get_functiondef, como se hizo en pgAdmin:

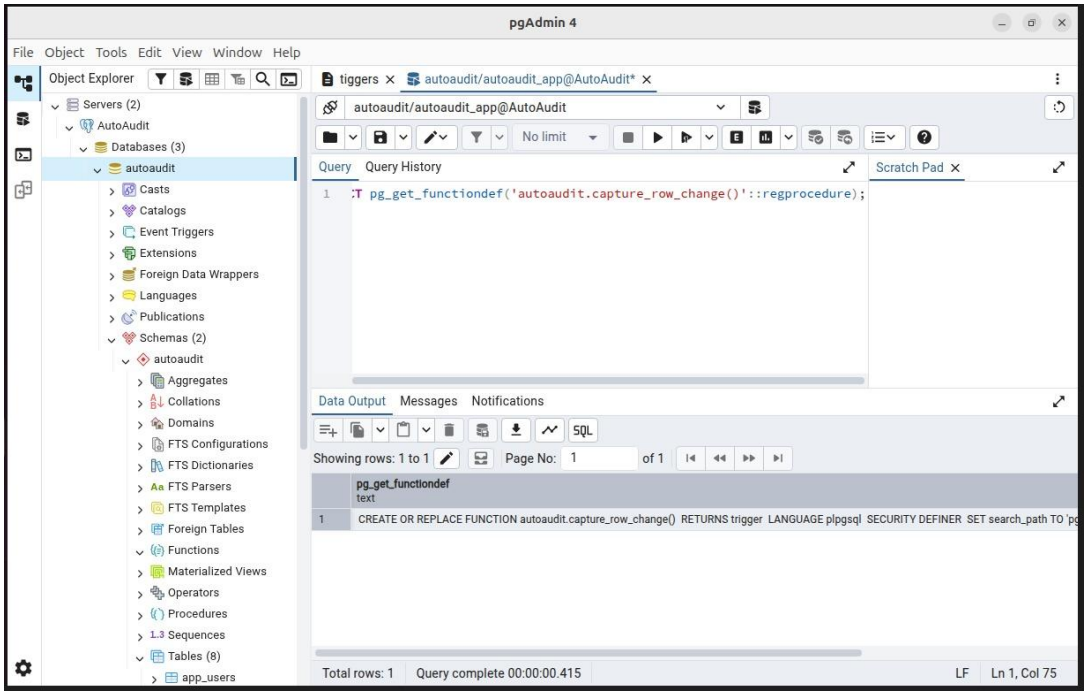


Figura 15. pgAdmin: consulta con pg_get_functiondef.

Para leerlo línea por línea se divide el texto con regexp_split_to_table. La captura muestra las primeras líneas: RETURNS trigger, LANGUAGE plpgsql, SECURITY DEFINER y SET search_path TO 'pg_catalog', 'autoaudit'; la función completa tiene 74 líneas.

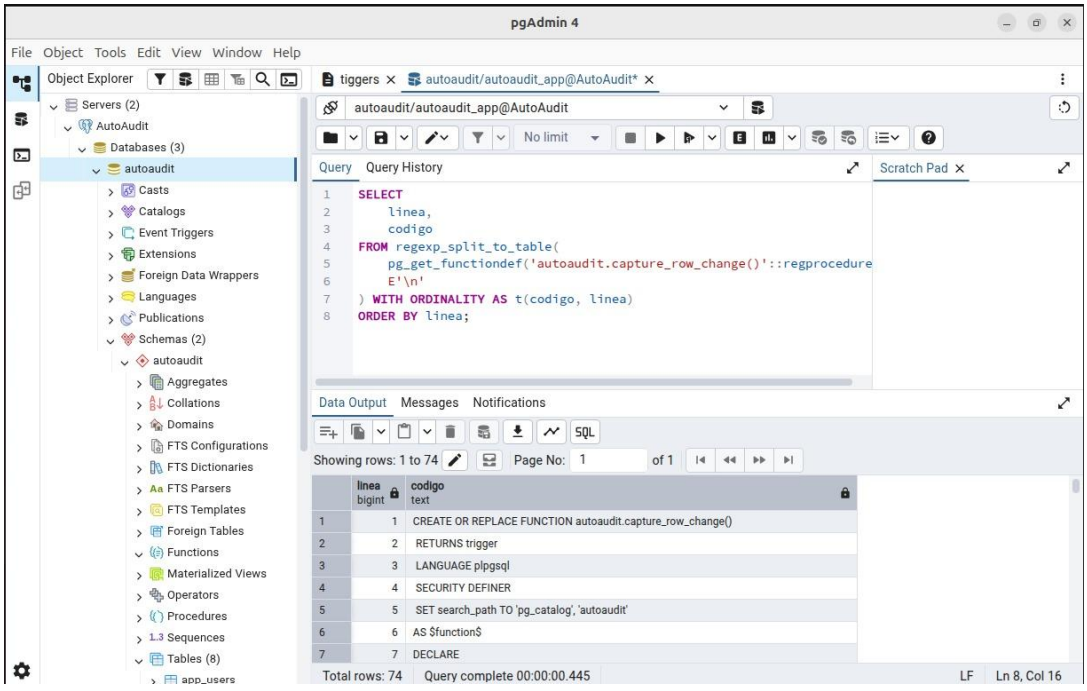


Figura 16. pgAdmin: código de capture_row_change() dividido en líneas.

11. Seguridad

11.1 Medidas implementadas

Tabla 23. Controles de seguridad.

Aspecto	Implementación	Observación
Autenticación	Formulario de login; `check_password_hash` contra `app_users.password_hash`; `session.clear()` antes de abrir la sesión.	Las contraseñas nunca se guardan en texto plano.
Autorización	Hook `before_request` por rol (`admin`, `auditor`).	La barra lateral también oculta las opciones de administración al auditor.
Inyección SQL	Valores con parámetros `%s`; identificadores con `sql.Identifier`; columnas validadas contra `information_schema`.	La vista previa del SQL en `/triggers` es solo informativa y no se ejecuta.
XSS	Autoescape de Jinja2 en las plantillas.	—
Secretos	`.env` fuera del control de versiones; `SECRET_KEY` configurable.	Debe cambiarse el valor por defecto.
Función de trigger	`SECURITY DEFINER` con `search_path` fijo.	El rol propietario debe ser de confianza.
Bitácora	La aplicación solo realiza consultas (`SELECT`) sobre la bitácora.	Un superusuario o el propietario puede alterarla.

12. Pruebas y validación

12.1 Preparación

1. Instalar el esquema (`01_schema.sql`), la tabla demo (`02_demo.sql`) y los usuarios (Anexo A).
2. Iniciar sesión como `admin` y abrir **Configuración**.
3. Elegir `public.empleados_demo`, marcar las columnas `nombre`, `puesto`, `salario` y `activo`, habilitar INSERT, UPDATE y DELETE, y pulsar «Guardar y generar trigger».
4. Ejecutar las sentencias de cada caso desde `psql` o pgAdmin conectado a la base `autoaudit`.

12.2 Casos de prueba

Tabla 24. Casos de prueba (los casos 1 a 6 provienen de tests/test_plan.md).

Caso	Acción	Sentencia / paso	Resultado esperado
1	INSERT	INSERT INTO public.empleados_demo(nombre, puesto, salario) VALUES ('Ana López','Analista',4200);	Fila en audit_log con operación INSERT y detalles con valor nuevo.
2	UPDATE	UPDATE public.empleados_demo SET salario=4500 WHERE nombre='Ana López';	Operación UPDATE; detalle de salario con anterior 4200 y nuevo 4500.
3	DELETE	DELETE FROM public.empleados_demo WHERE nombre='Ana López';	Operación DELETE; valores anteriores conservados.
4	Filtro por columna	En Bitácora, filtrar columna salario.	Solo aparecen los detalles de esa columna.
5	Desactivar	Desmarcar «Auditoría habilitada» y guardar.	Se elimina el trigger; los cambios posteriores no generan eventos.
6	Exportación	Pulsar «Exportar CSV».	Se descarga autoaudit_bitacora.csv con los resultados filtrados.
7	Contexto de usuario e IP	Ejecutar el bloque de la sección 12.3.	El evento queda con usuario ana.lopez e IP 192.168.1.10.
8	Login incorrecto	Ingresar una contraseña errónea.	Mensaje «Usuario o contraseña incorrectos.»
9	Acceso sin sesión	Abrir /bitacora sin iniciar sesión.	Redirección a /login.
10	Restricción del auditor	Iniciar como auditor y abrir /configurar.	Mensaje de falta de permisos y redirección al Dashboard.

12.3 Prueba del contexto de usuario e IP

```
BEGIN;
SELECT set_config('autoaudit.username',   'ana.lopez',    true),
       set_config('autoaudit.client_ip',   '192.168.1.10', true),
       set_config('autoaudit.user_agent',  'Prueba manual', true);
UPDATE public.empleados_demo SET puesto = 'Supervisor' WHERE nombre = 'Carlos
Pérez';
COMMIT;
```

12.4 Consultas de verificación

```
-- Últimos eventos
SELECT audit_id, occurred_at, schema_name, table_name, operation, app_username
FROM autoaudit.audit_log ORDER BY occurred_at DESC;

-- Detalle por columna
SELECT d.column_name, d.old_value, d.new_value
FROM autoaudit.audit_log_detail d ORDER BY d.detail_id DESC;

-- Evento con IP y detalle
SELECT l.audit_id, l.operation, l.app_username, w.ip_address,
       d.column_name, d.old_value, d.new_value
FROM autoaudit.audit_log l
LEFT JOIN autoaudit.audit_log_detail d  ON d.audit_id = l.audit_id
LEFT JOIN autoaudit.audit_workstations w ON w.workstation_id = l.workstation_id
ORDER BY l.audit_id DESC;
```

12.5 Pruebas de rendimiento con volumen creciente

El enunciado exige medir el costo de la auditoría con un volumen creciente de registros. Como cada fila auditada genera una fila de cabecera, una de estación y una por columna configurada, el costo es proporcional al número de filas modificadas; por ello se mide con 1 000, 10 000 y 100 000 filas. El procedimiento compara los mismos comandos sin auditoría y con auditoría.

```
CREATE TABLE IF NOT EXISTS public.rendimiento_demo (
   id    BIGSERIAL PRIMARY KEY,
   dato  TEXT NOT NULL,
   valor NUMERIC(10,2) NOT NULL
);
\timing on

-- Repetir con N = 1000, 10000 y 100000
INSERT INTO public.rendimiento_demo(dato, valor)
SELECT 'fila ' || g, g FROM generate_series(1, 10000) AS g;
UPDATE public.rendimiento_demo SET valor = valor + 1;
DELETE FROM public.rendimiento_demo;

-- Tamaño de la bitácora tras cada medición
SELECT COUNT(*) AS eventos,
      pg_size_pretty(pg_total_relation_size('autoaudit.audit_log')) AS audit_log,
      pg_size_pretty(pg_total_relation_size('autoaudit.audit_log_detail')) AS detalle
FROM autoaudit.audit_log;
```

1. Ejecutar el bloque anterior con la auditoría **deshabilitada** para `public.rendimiento_demo` y anotar los tiempos de `psql` (`Time:`).
2. En AutoAudit, configurar `public.rendimiento_demo` (columnas `dato` y `valor`; INSERT, UPDATE y DELETE) y guardar.
3. Repetir las mismas sentencias y anotar los tiempos y el tamaño de la bitácora.
4. Completar la tabla de resultados y concluir si el incremento es aceptable para el caso de uso.

Tabla 25. Resultados de la prueba de rendimiento (a completar con las mediciones del equipo).

Volumen (filas)	Operación	Sin auditoría (ms)	Con auditoría (ms)	Incremento (%)
1 000	INSERT	[ ]	[ ]	[ ]
1 000	UPDATE	[ ]	[ ]	[ ]
1 000	DELETE	[ ]	[ ]	[ ]
10 000	INSERT	[ ]	[ ]	[ ]
10 000	UPDATE	[ ]	[ ]	[ ]
10 000	DELETE	[ ]	[ ]	[ ]
100 000	INSERT	[ ]	[ ]	[ ]
100 000	UPDATE	[ ]	[ ]	[ ]

100 000	DELETE	[ ]	[ ]	[ ]

12.6 Evidencias de las pruebas realizadas

El equipo ejecutó los casos de inserción, actualización y eliminación sobre public.empleados_demo y verificó el resultado en la interfaz, en pgAdmin y en el archivo CSV exportado.

Tabla 26. Resultados observados en las pruebas.

Caso	Evidencia	Resultado observado
1. INSERT	Eventos 1 y 4 (figuras de 8.5 y 12.6)	Se registraron las inserciones de María López (id 2) y Pedro Ramírez (id 3), con una fila de detalle por columna y valor anterior vacío.
2. UPDATE	Evento 2	Se registró el cambio de salario de 4500.0 a 5000.0, con ctid_old y ctid_new.
3. DELETE	Evento 3	Se conservaron los valores de la fila eliminada (María López, salario 5000.0).
6. Exportación	Descarga y archivo CSV	Se descargó autoaudit_bitacora.csv y se abrió en LibreOffice Calc con las 11 columnas esperadas.

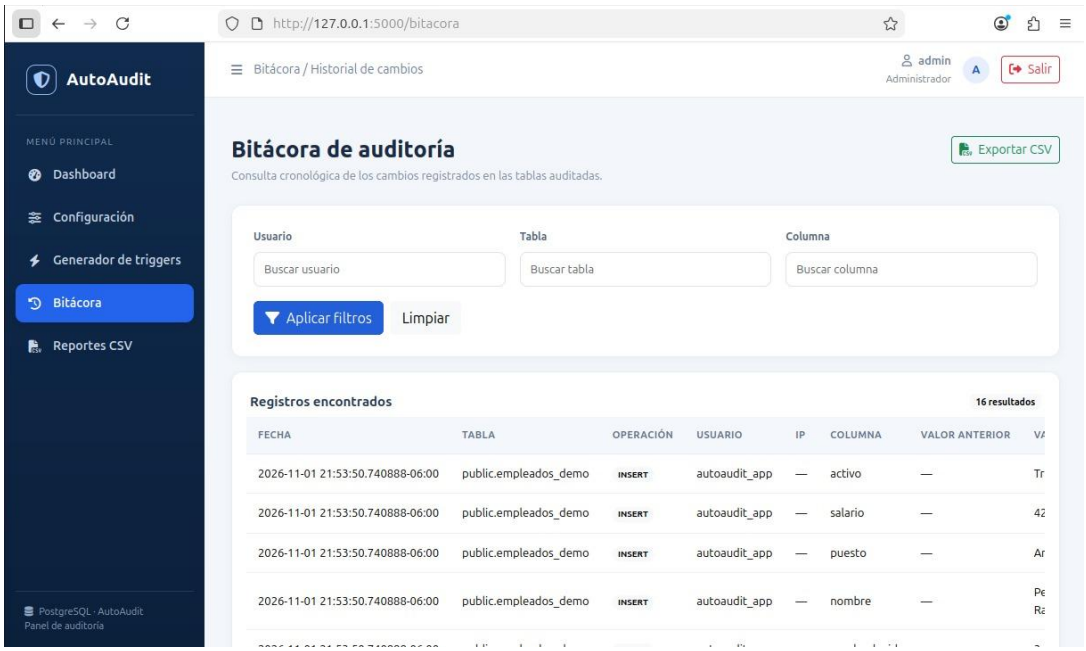


Figura 17. Bitácora de la interfaz con 16 resultados (4 eventos).

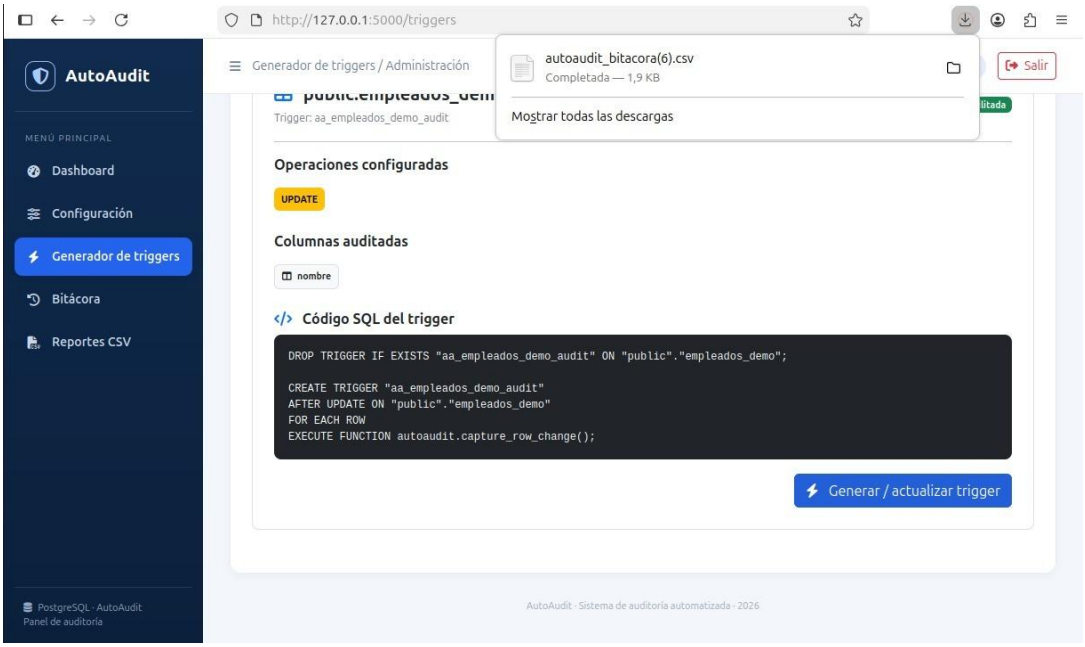
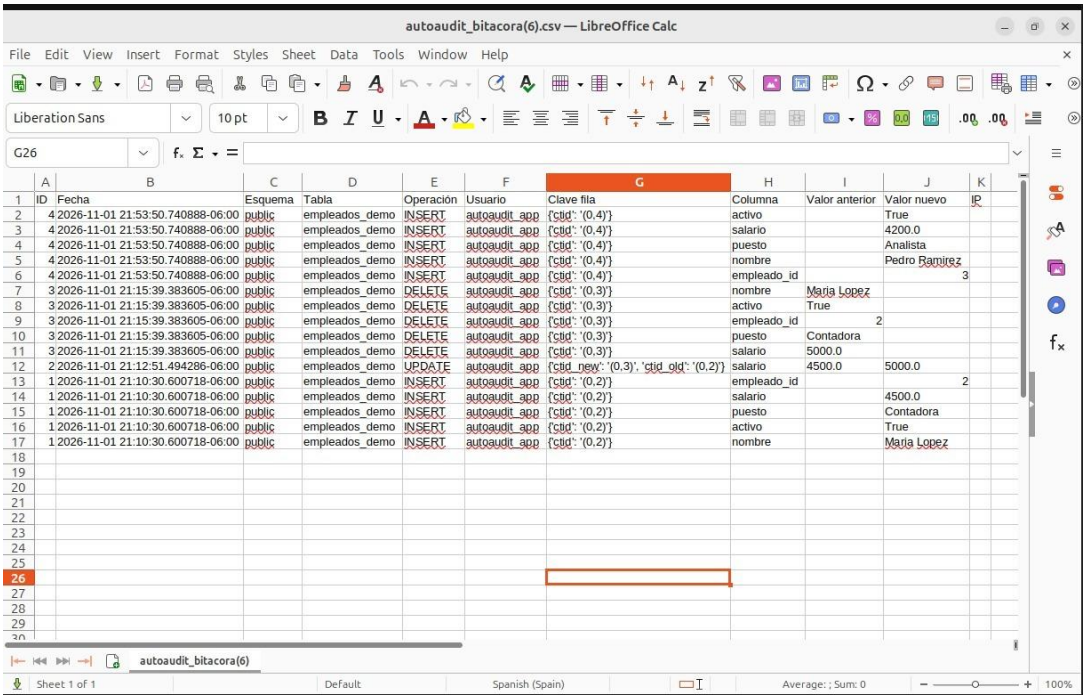


Figura 18. Descarga del archivo autoaudit_bitacora.csv desde el navegador.



	A	B	C	D	E	F	G	H	I	J	K
1	ID	Fecha	Esquema	Tabla	Operación	Usuario	Clave fila	Columna	Valor anterior	Valor nuevo	IP
2	4	2026-11-01 21:53:50.740888-06:00	public	empleados_demo	INSERT	autoaudit_app	{'ctid': '(0,4)'}	activo		True	
3	4	2026-11-01 21:53:50.740888-06:00	public	empleados_demo	INSERT	autoaudit_app	{'ctid': '(0,4)'}	salario		4200.0	
4	4	2026-11-01 21:53:50.740888-06:00	public	empleados_demo	INSERT	autoaudit_app	{'ctid': '(0,4)'}	puesto		Analista	
5	4	2026-11-01 21:53:50.740888-06:00	public	empleados_demo	INSERT	autoaudit_app	{'ctid': '(0,4)'}	nombre		Pedro Ramirez	
6	4	2026-11-01 21:53:50.740888-06:00	public	empleados_demo	INSERT	autoaudit_app	{'ctid': '(0,4)'}	empleado_id		3	
7	3	2026-11-01 21:15:39.383605-06:00	public	empleados_demo	DELETE	autoaudit_app	{'ctid': '(0,3)'}	nombre	Maria Lopez		
8	3	2026-11-01 21:15:39.383605-06:00	public	empleados_demo	DELETE	autoaudit_app	{'ctid': '(0,3)'}	activo	True		
9	3	2026-11-01 21:15:39.383605-06:00	public	empleados_demo	DELETE	autoaudit_app	{'ctid': '(0,3)'}	empleado_id	2		
10	3	2026-11-01 21:15:39.383605-06:00	public	empleados_demo	DELETE	autoaudit_app	{'ctid': '(0,3)'}	puesto	Contadora		
11	3	2026-11-01 21:15:39.383605-06:00	public	empleados_demo	DELETE	autoaudit_app	{'ctid': '(0,3)'}	salario	5000.0		
12	2	2026-11-01 21:12:51.494286-06:00	public	empleados_demo	UPDATE	autoaudit_app	{'ctid_new': '(0,3)', 'ctid_old': '(0,2)'}	salario	4500.0	5000.0	
13	1	2026-11-01 21:10:30.600718-06:00	public	empleados_demo	INSERT	autoaudit_app	{'ctid': '(0,2)'}	empleado_id		2	
14	1	2026-11-01 21:10:30.600718-06:00	public	empleados_demo	INSERT	autoaudit_app	{'ctid': '(0,2)'}	salario		4500.0	
15	1	2026-11-01 21:10:30.600718-06:00	public	empleados_demo	INSERT	autoaudit_app	{'ctid': '(0,2)'}	puesto		Contadora	
16	1	2026-11-01 21:10:30.600718-06:00	public	empleados_demo	INSERT	autoaudit_app	{'ctid': '(0,2)'}	activo		True	
17	1	2026-11-01 21:10:30.600718-06:00	public	empleados_demo	INSERT	autoaudit_app	{'ctid': '(0,2)'}	nombre		Maria Lopez	

Figura 19. Archivo CSV abierto en LibreOffice Calc.

Las capturas confirman el hallazgo 2 de la sección 13: la columna `Usuario` muestra `autoaudit_app` (usuario de la base de datos) y la columna `IP` está vacía en todos los eventos.

13. Mantenimiento

13.1 Respaldo y restauración

```
pg_dump -Fc -U autoaudit_app -h localhost autoaudit > autoaudit_$(date +%F).backup
pg_restore -U autoaudit_app -h localhost -d autoaudit_restaurada autoaudit_2026-11-03.backup
```

13.2 Consultas útiles

```
-- Triggers de auditoría existentes
SELECT tgrelid::regclass AS tabla, tgname
FROM pg_trigger WHERE tgname LIKE 'aa\_%' AND NOT tgisinternal;

-- Tamaño de la bitácora
SELECT pg_size_pretty(pg_total_relation_size('autoaudit.audit_log')) AS audit_log,
       pg_size_pretty(pg_total_relation_size('autoaudit.audit_log_detail')) AS detalle;

-- Eventos por tabla y operación
SELECT schema_name, table_name, operation, COUNT(*)
FROM autoaudit.audit_log GROUP BY 1,2,3 ORDER BY 4 DESC;
```

13.3 Auditar una tabla nueva

Después de crear una tabla, el administrador debe entrar a **Configuración**, seleccionarla y guardar la configuración; el trigger no se crea automáticamente al crear la tabla.

13.4 Actualización de dependencias

```
source .venv/bin/activate
pip install --upgrade -r requirements.txt
```

Tras actualizar, ejecutar el plan de pruebas de la sección 12 para confirmar que todo funciona.

13.5 Crecimiento de la bitácora

La bitácora almacena la fila completa antes y después, además del detalle por columna, por lo que crece rápidamente en tablas con mucha actividad. Defina una política de retención y archivado antes de eliminar registros; recuerde que borrar eventos reduce el valor probatorio de la auditoría.

14. Cronograma y calendario de control

El calendario de control del proyecto se resume en los cuatro hitos siguientes, que corresponden a las fases del cronograma del enunciado (Diseño, Implementación, Pruebas y Entrega final).

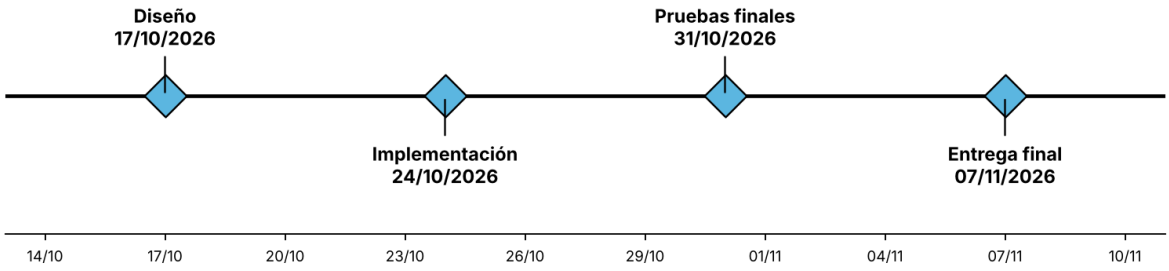


Figura 20. Hitos del proyecto.

Tabla 30. Calendario de control.

Fase	Actividades (según el cronograma del enunciado)	Fecha	Estado
Diseño	Análisis de requisitos. Diseño del modelo entidad-relación. Diseño de la interfaz.	17/10/2026	Realizado
Implementación	DDL de configuración. Diseño de consultas del diccionario de datos. Módulo de configuración. Programación del generador dinámico de triggers. Módulos de consulta y reporte.	24/10/2026	Realizado
Pruebas finales	Pruebas unitarias y de integración del sistema completo. Ajustes finales y documentación técnica completa.	31/10/2026	Realizado
Entrega final	Entrega del repositorio de fuentes, la documentación en PDF y las pruebas de validación.	07/11/2026	Programada

Anexo C. Glosario

Tabla 31. Términos utilizados.

Término	Definición
Auditoría	Registro sistemático de quién cambió qué dato, cuándo y desde dónde.
Trigger (disparador)	Objeto de la base de datos que ejecuta una función automáticamente ante un INSERT, UPDATE o DELETE.
PL/pgSQL	Lenguaje procedimental de PostgreSQL usado para escribir la función de auditoría.
JSONB	Tipo de dato de PostgreSQL para almacenar JSON en formato binario indexable.
DML	Lenguaje de manipulación de datos: INSERT, UPDATE, DELETE.
ctid	Identificador físico (página, posición) de una fila en PostgreSQL.
SECURITY DEFINER	Modo en que una función se ejecuta con los privilegios de su propietario.
Hash de contraseña	Resultado irreversible de aplicar un algoritmo criptográfico a una contraseña para poder verificarla sin guardarla.
CSRF	Ataque que fuerza a un usuario autenticado a enviar solicitudes no deseadas.
CDN	Red de distribución de contenido; aquí, servidores externos que entregan los estilos de la interfaz.

Anexo F. Manual de configuración y activación de auditoría por tabla

Procedimiento resumido para el administrador. El detalle de cada pantalla se encuentra en el Manual de usuario.

F.1 Activar la auditoría de una tabla

1. Iniciar sesión con un usuario de rol admin.
2. Abrir **Configuración**; elegir **Esquema** y **Tabla existente**; pulsar **Cargar tabla**. El sistema valida que la tabla y sus columnas existan en el catálogo.
3. Marcar las **columnas** que se auditarán (al menos una).
4. Marcar las **operaciones** INSERT, UPDATE y/o DELETE.
5. Dejar marcada **Auditoría habilitada** y pulsar **Guardar y generar trigger**.
6. Comprobar el mensaje «Trigger aa_<tabla>_audit creado correctamente.» y revisar el resultado en **Generador de triggers** (Figura de la sección 9.5).

7. Verificar en la base de datos con las consultas siguientes.

Ejemplo real del equipo: se configuró public.empleados_demo solo para la operación UPDATE y la columna nombre. El Generador de triggers muestra el resultado:

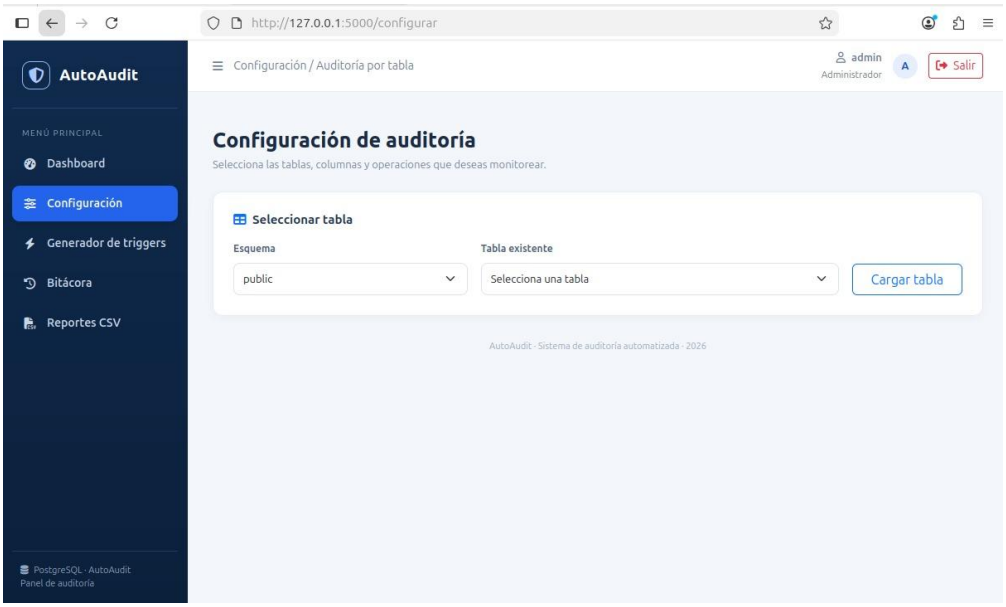


Figura 21. Pantalla de Configuración para seleccionar esquema y tabla.



Figura 22. Generador de triggers: configuración UPDATE sobre la columna nombre.

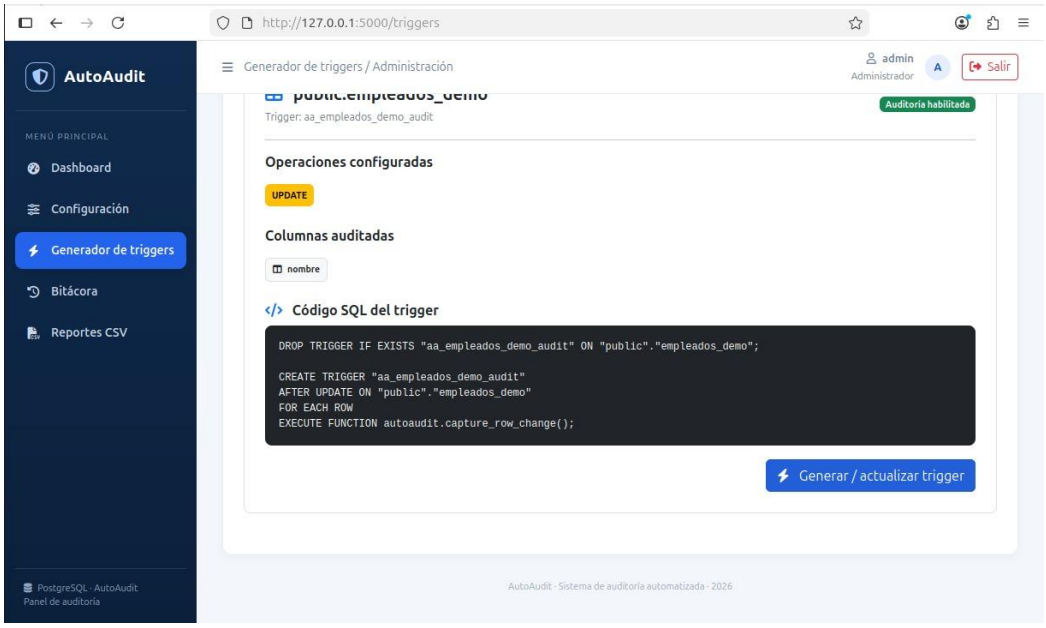


Figura 23. Generador de triggers: SQL generado para operación UPDATE.

```
SELECT * FROM autoaudit.audit_table_config WHERE table_name =
'empleados_demo';
SELECT column_name, enabled FROM autoaudit.audit_column_config
WHERE config_id = (SELECT config_id FROM autoaudit.audit_table_config
                   WHERE table_name = 'empleados_demo');
SELECT tgname, tgenabled FROM pg_trigger WHERE tgname =
'aa_empleados_demo_audit';
```

F.2 Modificar la configuración

Cargar nuevamente la tabla, cambiar columnas u operaciones y volver a guardar. El trigger se reemplaza con la nueva configuración.

F.3 Desactivar la auditoría

Desmarcar **Auditoría habilitada** (o dejar sin operaciones) y guardar. El trigger se elimina; la configuración y el historial permanecen. Para reactivar, marcar de nuevo la casilla y guardar.

F.4 Auditar una tabla creada después

El sistema no detecta tablas nuevas de forma automática en cuanto a triggers: después de crear la tabla, debe repetirse el procedimiento F.1 para ella.